
Allow initializing aggregates from a parenthesized list of values

Abstract

This paper proposes allowing initializing aggregates from a parenthesized list of values; that is, `Aggr(val1, val2)` would mean the same thing as `Aggr{val1, val2}`, except that narrowing conversions are allowed. This is a language fix for the problem illustrated in [N4462](#).

Revision history

- P0960R0 - initial version
- P0960R1 - wording created based on EWG feedback from Jacksonville 2018
- P0960R2 - proposal changed to model an invented constructor call rather than textual transformation to a braced list

Jacksonville 2018 discussion feedback

- Do we want more work in the direction depicted in p0960r0? 12 | 11 | 7 | 1 | 1
- Restrict the change to just dependent types? 1 | 1 | 2 | 13 | 14
- Should designated initializers be supported? 1 | 3 | 12 | 11 | 3
- Should raw arrays be supported? 4 | 14 | 11 | 2 | 1
- Incorporate ban brace-initializing an object if it would be plausible that it would use a deleted constructor? 11 | 14 | 5 | 0 | 3

Revision R1 implements supporting parenthesized initialization for aggregates including arrays, without support for designated initializers. The matter of initialization and deleted constructors is handled by separate paper(s).

Rapperswil 2018 discussion feedback

From the initial EWG discussion:

- P0960R1 as presented: 1 | 2 | 14 | 20 | 3
- P0960R1 but allowing narrowing conversions: 10 | 15 | 13 | 5 | 0

After CWG review and feedback:

- Accept CWG's recommendation to switch to a "as if by constructor" model: 18 | 15 | 9 | 2 | 1

- Make `std::vector<std::array<int, 3>>().emplace_back(1, 2, 3)` work via language specification:
3 | 1 | 20 | 11 | 11

After further CWG feedback and EWG discussion:

- Extend lifetimes of temporaries? 1 | 0 | 21 | 11 | 1
- Should evaluation occur in-order? 5 | 12 | 10 | 7 | 1
- Make aggregate initialization preferable over conversions? 1 | 2 | 15 | 11 | 3
- Make it behave *exactly* like a constructor, e.g. indeterminate evaluation order, non-extension of lifetime? 9 | 12 | 8 | 3 | 1
- Should the constructor be non-explicit? 3 | 2 | 12 | 9 | 4

This revision changes the mental model from the original “literal rewrite to a braced list” to “as if a synthesized, explicit constructor with appropriate *mem-initializers* was called”. This has the effect of allowing narrowing conversions in the parenthesized list, even when narrowing conversions would be forbidden in the corresponding braced list syntax. It also clarifies the non-extension of temporary lifetimes of temporaries bound to references, the absence of brace elision, and the absence of a well-defined order of evaluation of the arguments.

During the discussion, it was suggested by CWG that we should even break an existing corner case: Given `struct A; struct C { operator A(); }; struct A { C c; };,` the declaration `A a(c);` currently invokes `C`’s conversion function to `A`. It was suggested to change this behaviour to use the (arguably better-matching) aggregate initialization in this case, i.e. to behave like `A a{c};` and not like `A a = c;`. There was, however, no consensus to pursue this direction, and the proposal remains a pure extension at this point.

Design principles

- Any existing meaning of `A(b)` should not change.
- Parenthesized initialization and braced-initialization should be as similar as possible, but as distinct as necessary to conform with the existing mental models of braced lists and parenthesized lists.
- In particular, we have the following differences:
 - For an aggregate `A`, `A{b}` will not convert `b` to `A`, unless `b` is of type `A` or type derived from `A`. `A(b)` will convert even in other cases. This difference applies only to non-arrays.
 - Aggregate-initialization `A{x, y, x}` forbids narrowing conversions, while the new syntax `A(x, y, z)` does allow narrowing.
 - A temporary bound to a reference member of the aggregate via braces has its lifetime extended, whereas no extension happens when the temporary is passed via round parentheses.
 - The elements of the braced list are evaluated in order, whereas parenthesized lists usually carry the connotation of indeterminate evaluation order (like for a function call). However, the evaluation order will necessarily be definite in this situation because aggregate elements are initialized in order, and directly from the corresponding initializer elements.

Wording

In [dcl.init]/17, delete item (17.5):

~~Otherwise, if the destination type is an array, the program is ill-formed.~~

In [dcl.init]/(17.6.2), edit as follows:

Otherwise, if the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same class as, or a derived class of, the class of the destination, constructors are considered. The applicable constructors are enumerated (16.3.1.3), and the best one is chosen through overload resolution (16.3). The constructor so selected is called to initialize the object, with the initializer expression or expression-list as its argument(s). If no constructor applies and the destination type is not an aggregate, or the overload resolution is ambiguous, the initialization is ill-formed.

After [dcl.init]/(17.6.3) and before [dcl.init]/(17.7), insert a new top-level bullet as follows:

If the destination type is an aggregate and the initializer is a parenthesized *expression-list* and no viable constructor was found above, the object is initialized as follows. Let $e_1, \dots, e_n$ be the elements the aggregate [dcl.init.aggr], and let $\tau_1, \dots, \tau_n$ be their respective declared types. Let $x_1, \dots, x_k$ be the elements of the *expression-list*. If k is greater than n , the program is ill-formed. For each integer $1 \leq i \leq n$, let v_i be the empty token sequence, unless the destination type is a class type and e_i has a default member initializer [class.mem], in which case v_i consists of the same tokens as that default member initializer.

- If the destination is an array of type $U[N]$, then for each integer $0 \leq i < k$, the i^{th} element is copy-initialized with x_{i+1} , and for each integer $k \leq i < N$, the i^{th} element is direct-list-initialized with $\{\}$.
- Otherwise, the destination type is a (possibly cv-qualified) class type A . Given an invented, public constructor of the form

`explicit A($\tau_1\&\& t_1, \dots, \tau_k\&\& t_k$) : $e_1(\text{std}::\text{forward}<\tau_1>(t_1)), \dots,$
 $e_k(\text{std}::\text{forward}<\tau_k>(t_k), e_{k+1}(v_{k+1}), \dots, e_n(v_n)) \{\}$`

that is a constexpr constructor [dcl.constexpr] if it satisfies the requirements for a constexpr constructor, then if the expression $A(x_1, \dots, x_k)$ is ill-formed, the program is ill-formed. Otherwise, the object is initialized as if by $A(x_1, \dots, x_k)$.

[Note: Designators are not permitted. By contrast with direct-list-initialization, narrowing conversions [dcl.list.init] are permitted, the order of evaluation of the list elements is indeterminate, a temporary object [class.temporary] bound to a reference does not have its lifetime extended, and there is no brace elision. [Example:

```
struct A {  
    int a;  
    int&& r;  
};  
  
int f();  
  
A a1{1, f()};    // OK
```

```
A a2{1.0, 1};      // Error, narrowing conversion  
A a2(1.0, f());    // well-formed, but dangling reference
```

— end example] — end note]

Drafting note: the note in the above wording can easily be left out - the grammar of a parenthesized initialization does not allow designators. The note is added just for explanatory clarity.